Práctica 3: Aprendizaje por Refuerzo

Izan Cuesta Corbí
DNI: 48804930B
Universidad de Alicante – Escuela Politécnica Superior
Aprendizaje Avanzado – Curso 2025/2026
Email: icc87@alu.ua.es

Resumen

En esta práctica se aborda la resolución del entorno `LunarLander-v3`, formalizado como un Proceso de Decisión de Markov (MDP) de estados continuos y acciones discretas. Para ello, se implementaron tres métodos: una política aleatoria, el algoritmo REINFORCE *vanilla* (junto a una variante con mejoras opcionales de entropía) y REINFORCE con línea base (Actor-Crítico). Se exploró el ajuste de hiperparámetros clave como el factor de descuento, el coeficiente de exploración y, crucialmente, el uso de tasas de aprendizaje asimétricas entre actor y crítico para garantizar estimaciones fiables. Los resultados empíricos en la fase de evaluación revelan que el agente aleatorio (-178.19) y el modelo básico puramente estricto (-102.04) son incapaces de resolver el problema, colapsando en óptimos locales de caída libre. Las mejoras de normalización elevaron el rendimiento a 90.31 puntos, pero fue la arquitectura de línea base la que marcó la diferencia estructural definitiva. Al reducir drásticamente la varianza del gradiente, el modelo Actor-Crítico estabilizó la convergencia y obtuvo una recompensa media sobresaliente de 188.56 puntos, dominando la maniobra de aterrizaje y demostrando la eficacia del Aprendizaje por Refuerzo Profundo.

Palabras clave: Aprendizaje por refuerzo, REINFORCE, Gradientes de políticas, Línea base, Actor-Crítico, LunarLander, Gymnasium.

Índice

1. Ejercicio 1: Política Aleatoria **3**
1.1. Implementación del Bucle de Interacción . . . 3
1.2. Resultados . . . 3
1.3. Análisis . . . 3

2. Ejercicio 2: REINFORCE **4**
2.1. Fase A: Entrenamiento . . . 4
2.1.1. Hiperparámetros y Justificación . . . 4
2.1.2. Resultados del Entrenamiento . . . 5
2.2. Fase B: Evaluación . . . 6
2.3. Tarea Opcional: Mejoras sobre REINFORCE . . . 7
2.3.1. Normalización de Retornos . . . 7
2.3.2. Bonus de Entropía . . . 8
2.3.3. Fase A: Entrenamiento . . . 8
2.3.4. Fase B: Evaluación . . . 9

3. Ejercicio 3: REINFORCE con Línea Base **10**
3.1. Fase A: Entrenamiento . . . 10
3.1.1. Hiperparámetros y Justificación . . . 10
3.1.2. Resultados del Entrenamiento . . . 11
3.2. Fase B: Evaluación . . . 12

4. Comparación Global **13**

5. Conclusiones **14**

1. Ejercicio 1: Política Aleatoria

1.1. Implementación del Bucle de Interacción

Se implementó el bucle básico de interacción agente-entorno durante 100 episodios consecutivos, donde el agente selecciona acciones de forma puramente aleatoria mediante `env.action_space.sample()`.

Para implementar el agente aleatorio, he inicializado el entorno `LunarLander-v3` de Gymnasium utilizando el parámetro `render_mode="rgb_array"`. Esto permite capturar los fotogramas de la simulación internamente para generar el GIF sin necesidad de abrir ventanas gráficas. Para registrar las métricas del experimento, que consta de 100 episodios, he instanciado dos listas dinámicas destinadas a almacenar la recompensa total acumulada y la duración (en pasos de tiempo) de cada intento.

Durante el bucle de interacción, el entorno y los contadores se reinician al comienzo de cada episodio. Mientras el episodio permanece activo, el agente selecciona acciones de forma puramente al azar utilizando el método `env.action_space.sample()`. Tras aplicar cada acción en el entorno, se obtiene la recompensa inmediata y se acumula al total, repitiendo el proceso hasta alcanzar un estado terminal (aterrizaje, colisión o límite de tiempo). Al concluir, los totales se guardan en las listas correspondientes, procesándose al final de los 100 episodios para calcular las medias, desviaciones estándar y generar las gráficas visuales.

1.2. Resultados

Tabla 1: Estadísticas de la política aleatoria (100 episodios)

Métrica	**Media**	**Desv. Estándar**
Recompensa acumulada	-178.19	110.27
Duración (pasos)	89.23	20.39

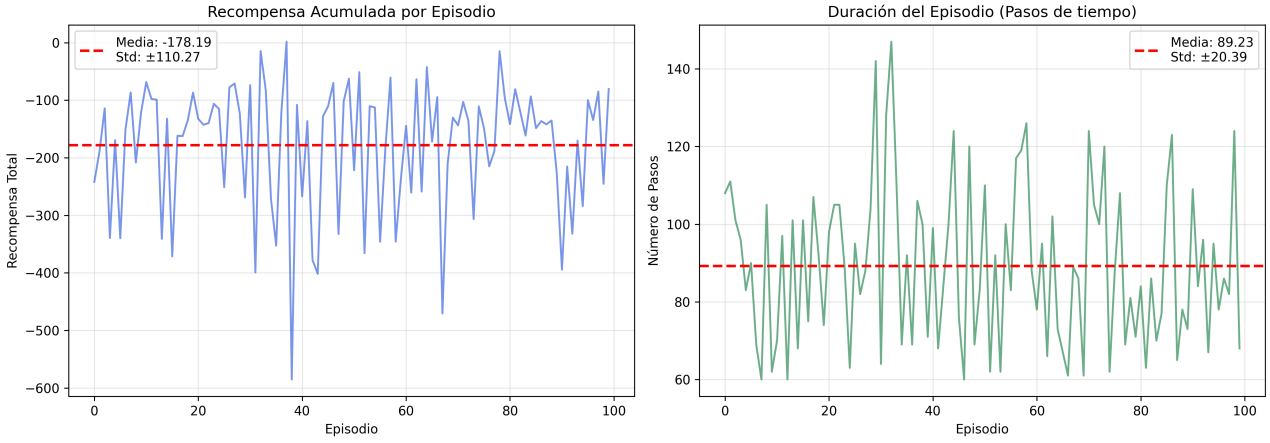


Figura 1: Evolución de métricas – política aleatoria (100 episodios)

1.3. Análisis

La política aleatoria resulta completamente ineficaz en el entorno `LunarLander-v3` debido a la complejidad de las dinámicas físicas involucradas. Aterrizar con éxito requiere una secuencia coordinada y precisa de acciones temporales para contrarrestar la gravedad, anular el momento angular y dirigir la nave hacia las coordenadas exactas de la plataforma. Al seleccionar acciones al azar, el agente carece de cualquier noción de planificación a largo plazo, provocando que sus decisiones se anulen entre sí o empeoren drásticamente situaciones de peligro.

Observando la simulación generada, se evidencian múltiples comportamientos erráticos que impiden un aterrizaje seguro, destacando los giros descontrolados, las fases de caída libre y un uso

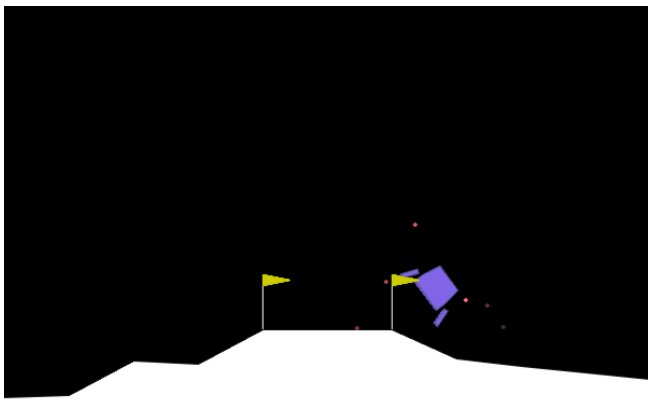


Figura 2: Episodio representativo – agente aleatorio

ineficiente de los motores. Específicamente, tal y como se ilustra en la captura, se aprecia cómo la nave acaba colisionando de forma catastrófica directamente contra la bandera derecha que delimita la zona objetivo. El módulo lunar se muestra severamente ladeado y activando los propulsores de forma asimétrica y sin sentido a escasos metros del suelo. Este uso contraproducente del sistema de propulsión provoca que, en lugar de amortiguar el descenso, la nave pierda por completo su centro de gravedad y se estrelle violentamente sin ningún tipo de control sobre su orientación.

Este comportamiento destructivo se refleja con claridad en las métricas obtenidas en la evaluación. La recompensa media tras 100 episodios es de -178.19 (con una alta desviación estándar de ±110,27), producto de las severas penalizaciones que impone el entorno al estrellar la nave o salirse de los límites de la pantalla. Teniendo en cuenta que el entorno requiere una media de 200 puntos positivos para considerarse resuelto, existe una enorme brecha teórica y práctica de más de 378 puntos. Finalmente, la duración media de los episodios, de apenas 89.23 pasos de tiempo, confirma empíricamente que la nave es incapaz de mantenerse en vuelo prolongado y colisiona a los pocos segundos tras el inicio de cada intento.

2. Ejercicio 2: REINFORCE

2.1. Fase A: Entrenamiento

2.1.1. Hiperparámetros y Justificación

Para determinar la configuración definitiva detallada en la Tabla 2, he ejecutado una búsqueda empírica evaluando siete variaciones estructurales durante una fase temprana de 300 episodios. Este enfoque permite aislar y comprender el impacto individual de cada hiperparámetro en la dinámica de aprendizaje del algoritmo REINFORCE puro.

En primer lugar, la elección del optimizador resulta ser el factor más crítico. Se seleccionó **Adam** frente a **SGD** debido a la propia naturaleza matemática del algoritmo. REINFORCE actualiza la política basándose en trayectorias completas muestreadas mediante técnicas de Monte Carlo, lo que genera gradientes con una varianza altísima. La prueba empírica con SGD demostró que el Descenso de Gradiente Estocástico tradicional es incapaz de lidiar con este nivel de ruido, provocando un colapso absoluto en el rendimiento (alcanzando una puntuación crítica de -548.72). Adam soluciona este problema adaptando dinámicamente la tasa de aprendizaje para cada parámetro utilizando momentos de primer y segundo orden.

Respecto a la **tasa de aprendizaje (α)**, la experimentación revela una trampa habitual en el Aprendizaje por Refuerzo. Al probar una tasa alta de 0.01, el modelo pareció aprender a sobrevivir más rápido inicialmente (obteniendo la mejor puntuación temporal de -130.79 a los 300 episodios). Sin embargo, extrapolando estos saltos agresivos en la superficie de pérdida a 1000 episodios, el modelo sufre de inestabilidad y divergencia (olvido catastrófico), destruyendo su propia política de vuelo. Una tasa demasiado baja (0.0001) es excesivamente lenta (-149.51). Por tanto, el valor de 0.001 se fija como

Tabla 2: Hiperparámetros seleccionados para REINFORCE

Hiperparámetro	**Valor**	**Justificación**
Tasa de aprendizaje (α)	0.001	Proporciona actualizaciones de pesos estables. Valores mayores inducen divergencia a largo plazo y valores menores ralentizan en exceso la convergencia.
Factor de descuento (γ)	0.99	Priorizar recompensas futuras (el aterrizaje exitoso) frente a la penalización inmediata por gasto de combustible.
Neuronas ocultas (`n_hidden`)	128	Ofrece un equilibrio óptimo entre capacidad de representación del entorno y eficiencia computacional, sin riesgo de sobreajuste temprano.
Optimizador	Adam	Esencial para manejar la alta varianza de los gradientes de REINFORCE mediante la adaptación dinámica de la tasa de aprendizaje.
Número de episodios	1000	Horizonte suficiente para evaluar la capacidad de supervivencia y estabilización del agente antes de introducir técnicas avanzadas como la línea base.

el punto de equilibrio idóneo para un progreso monótono y estable.

El **factor de descuento (γ)** dicta el horizonte temporal de planificación del agente. Al reducir este valor a 0.50 durante las pruebas empíricas, el comportamiento del agente se vuelve puramente cortoplacista. Dado que el entorno `LunarLander-v3` penaliza el uso del motor en cada instante temporal pero premia con +100 puntos el aterrizaje final, un γ bajo provocó que el agente ignorara por completo la recompensa del aterrizaje y optara por apagar los motores para minimizar el gasto actual, dejándose caer en picado. Esto justifica la necesidad de un γ muy cercano a 1 (0.99) para propagar correctamente el valor del éxito final a las acciones iniciales del vuelo.

Finalmente, el ajuste de la **arquitectura neuronal** (probando 64, 128 y 256 neuronas ocultas) nos da resultados estadísticamente equivalentes (en torno a -140 puntos). Esto demuestra que el cuello de botella del algoritmo Vanilla no es la capacidad expresiva de la red, sino la ineficiencia muestral intrínseca del método, validando la elección de 128 neuronas como el diseño más eficiente a nivel computacional.

2.1.2. Resultados del Entrenamiento

Tabla 3: Estadísticas del entrenamiento REINFORCE (1000 episodios)

Métrica	**Media**	**Desv. Estándar**
Recompensa acumulada	-144.25	67.70
Duración (pasos)	80.53	17.83

A partir de las curvas de aprendizaje generadas y los registros estadísticos del entrenamiento, se observa una ausencia total de convergencia hacia comportamientos exitosos a lo largo de los 1000 episodios. A diferencia de lo que se esperaría en un aprendizaje funcional, la masa de recompensas se encuentra sumida y estancada de forma permanente en valores fuertemente negativos, promediando -144.25 puntos globales. La gráfica no muestra ninguna tendencia creciente sostenida hacia el eje positivo, lo que indica empíricamente que el agente puramente básico es incapaz de asimilar la dinámica de la nave ni de optimizar la política de forma efectiva.

Analizando la progresión temporal y la gráfica de duración de los episodios, resulta evidente que el agente ha colapsado en un óptimo local destructivo desde las fases iniciales. La curva de pasos de tiempo revela una duración media de apenas 80.53 pasos, sin mostrar intentos de supervivencia

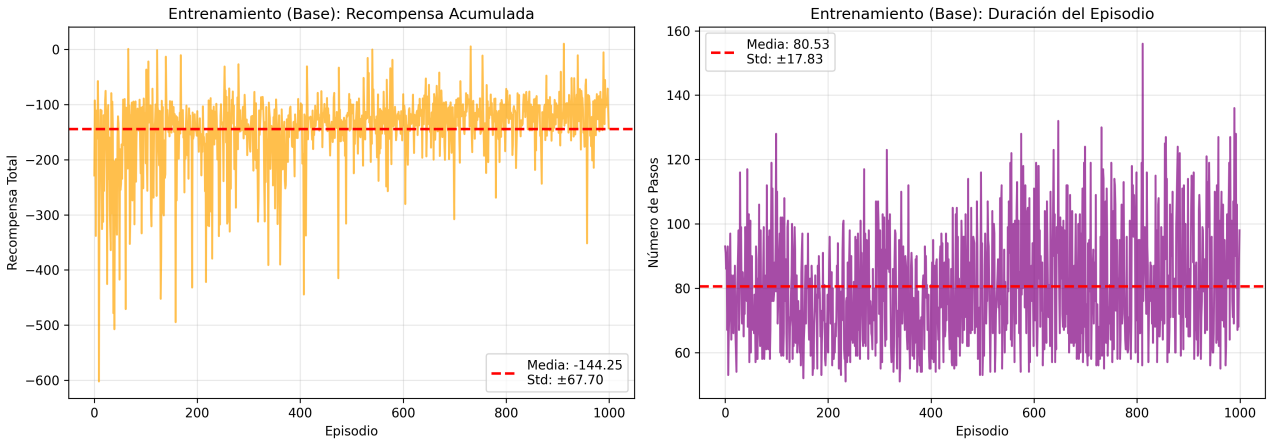


Figura 3: Curvas de entrenamiento – REINFORCE (1000 episodios)

prolongada. Al carecer de un mecanismo explícito que fomente la exploración constante (como el bonus de entropía), la red neuronal concluye prematuramente que la estrategia matemática más eficiente para evitar la pérdida de puntos por el consumo de combustible es no activar los motores. En consecuencia, el agente opta por dejarse caer en picado sistemáticamente, asumiendo la penalización ineludible por colisión como el mal menor.

Tal y como dicta la teoría del algoritmo REINFORCE en su forma más estricta, el proceso de actualización sufre de graves problemas de asignación de crédito. Al carecer de la normalización de retornos, el agente percibe todas las trayectorias como valores absolutos negativos. Esto provoca que el gradiente penalice y empuje a la baja las probabilidades de prácticamente todas las acciones ejecutadas, imposibilitando que el modelo logre distinguir entre una maniobra de estabilización moderadamente útil y un error catastrófico. La desviación estándar de ±67,70 no representa intentos audaces de aterrizaje, sino el ruido intrínseco de un modelo estocástico que se estrella repetidamente bajo condiciones iniciales ligeramente distintas.

Como consecuencia directa de esta inestabilidad estructural y la incapacidad de procesar recompensas relativas, el agente fracasa a la hora de acercarse a la marca de los 200 puntos, manteniéndose en un estado de inoperancia durante la totalidad del entrenamiento. Estos resultados demuestran de forma visual y numérica que el algoritmo REINFORCE clásico resulta ineficiente para resolver entornos continuos con penalizaciones severas como `LunarLander-v3`. Esta profunda deficiencia práctica justifica plenamente la necesidad de incorporar técnicas de estabilización algorítmica, tales como la normalización, la entropía y, de manera fundamental, la introducción de una arquitectura Actor-Crítico provista de una línea base (*baseline*).

2.2. Fase B: Evaluación

Tabla 4: Estadísticas de evaluación – REINFORCE (100 episodios)

Métrica	**Media**	**Desv. Estándar**
Recompensa acumulada	-102.04	36.74
Duración (pasos)	90.61	19.89

Al contrastar los resultados de la fase de evaluación del agente entrenado con REINFORCE Vanilla frente al agente aleatorio del Ejercicio 1, se observa una mejora puramente numérica, aunque prácticamente nula a nivel operativo. La recompensa media ha pasado de -178.19 (política aleatoria) a -102.04. Si bien esto supone una recuperación matemática de aproximadamente 76 puntos, no indica que el agente haya aprendido a volar. Al mantener una duración media extremadamente corta de 90.61 pasos, queda en evidencia que el agente únicamente ha aprendido a dejarse caer para minimizar la penalización por el uso continuo de los propulsores, aceptando la colisión inminente como el resultado

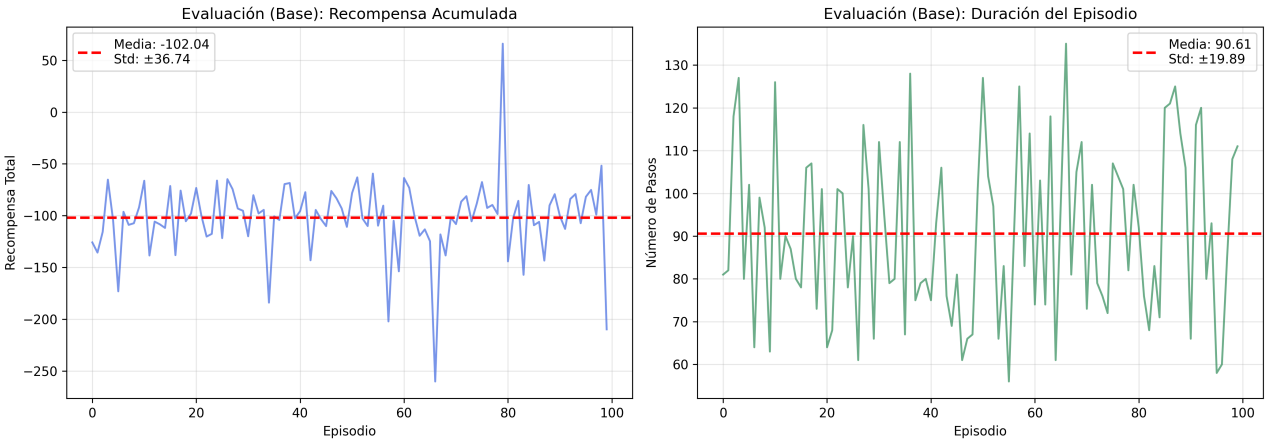


Figura 4: Evaluación – REINFORCE (100 episodios)

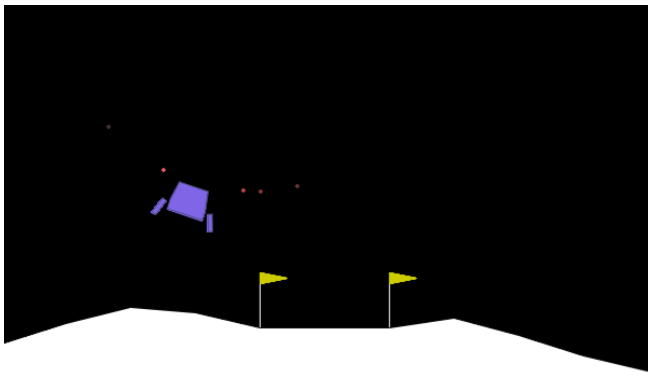


Figura 5: Episodio representativo – agente REINFORCE

óptimo dentro de su limitada exploración.

Este análisis cuantitativo se confirma de manera visual al observar el comportamiento del agente en la simulación (GIF). Lejos de mostrar un vuelo controlado, el agente no logra estabilizar la nave, no corrige la trayectoria de descenso hacia la zona de aterrizaje y es incapaz de posarse suavemente. Tal y como captura el fotograma de la evaluación, el módulo lunar adquiere ángulos de inclinación críticos y activa los propulsores de manera errática o tardía. En consecuencia, el aterrizaje consistente es inexistente; la inmensa mayoría de los episodios terminan en colisiones catastróficas (*crashes*) contra la superficie lunar.

Por consiguiente, el agente fracasa de forma rotunda a la hora de superar el umbral de los 200 puntos. Al observar la gráfica de recompensas de evaluación, la línea de tendencia se mantiene plana y fuertemente anclada en torno a la marca de los -100 puntos, valor que coincide exactamente con la penalización directa impuesta por el entorno al estrellarse. Aunque la gráfica muestra un único pico anecdótico que logra rozar los 50 puntos positivos (probablemente fruto de unas condiciones iniciales extremadamente favorables donde la nave cayó de pie por pura estocasticidad), la consistencia es nula. Estos datos certifican de manera definitiva que el algoritmo REINFORCE en su arquitectura básica es inoperante para resolver el entorno sin la asistencia de mejoras algorítmicas.

2.3. Tarea Opcional: Mejoras sobre REINFORCE

2.3.1. Normalización de Retornos

La normalización de retornos estandariza el tensor de retornos descontados (G_t) de cada episodio para que tenga media cero y desviación estándar uno, restando la media y dividiendo por la desviación estándar (más un ϵ para la estabilidad numérica).

En entornos como `LunarLander-v3`, un agente sin normalizar recibe exclusivamente retornos negativos al inicio del entrenamiento. Esto empuja a la baja las probabilidades de todas las acciones, provocando estancamientos en óptimos locales (como dejarse caer en picado para evitar consumir combustible). La normalización soluciona este problema transformando las recompensas absolutas en relativas: las acciones "menos malas"que la media (ej. sobrevivir más tiempo) reciben automáticamente un retorno positivo, fomentando su repetición.

Empíricamente, al observar las curvas de entrenamiento, esta operación elimina el colapso inicial del algoritmo Vanilla. Se reduce drásticamente la varianza y el ruido del gradiente, logrando una progresión de aprendizaje mucho más suave y dirigida que escala de forma rápida hacia recompensas positivas.

2.3.2. Bonus de Entropía

El bonus de entropía se implementa restando la entropía de la distribución de probabilidad de las acciones a la función de pérdida de la política. Dado que el optimizador busca minimizar dicha pérdida, esta resta matemática obliga a la red neuronal a maximizar la entropía, penalizando el exceso de confianza en sus decisiones.

Para esta ejecución, se fijó un coeficiente de entropía $\beta = 0{,}01$. Este valor es lo suficientemente alto para incentivar la exploración en las fases tempranas, pero lo bastante moderado como para no opacar la señal de la recompensa una vez que el agente comienza a dominar el aterrizaje.

Su efecto empírico es vital para evitar la convergencia prematura. Sin este bonus, el agente tiende a colapsar en óptimos locales destructivos (como apagar los motores y estrellarse rápidamente para no gastar combustible, o quedarse flotando indefinidamente). Al forzar a la red a mantener un pequeño grado de incertidumbre, el agente se ve obligado a probar nuevas combinaciones de propulsión. En las curvas de entrenamiento, esto se traduce en la capacidad del modelo para escapar del estancamiento inicial (-100 puntos) y explorar de forma activa hasta descubrir la política de vuelo óptima.

2.3.3. Fase A: Entrenamiento

Tabla 5: Estadísticas del entrenamiento REINFORCE MEJORADO (1000 episodios)

Métrica	**Media**	**Desv. Estándar**
Recompensa acumulada	-4.19	106.65
Duración (pasos)	415.45	392.38

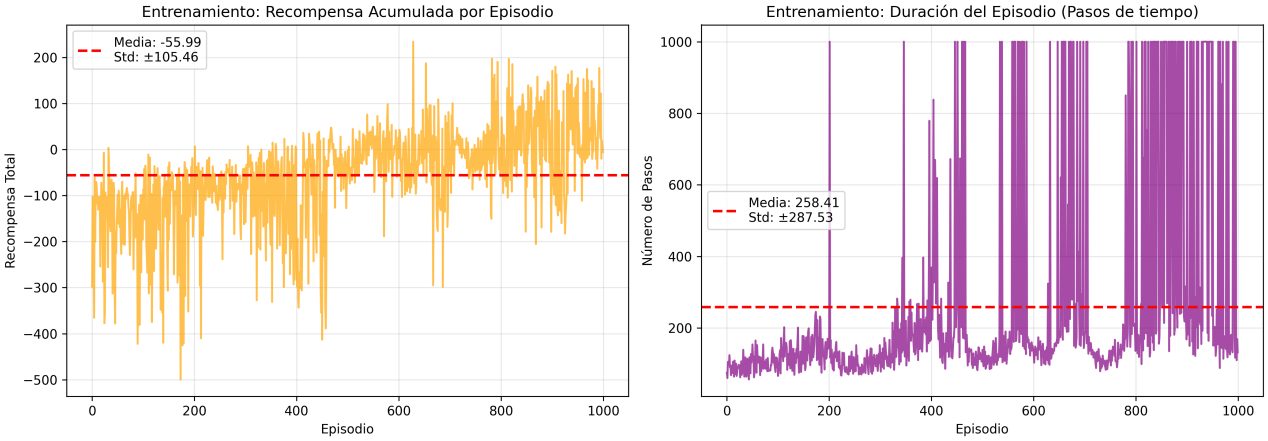


Figura 6: Curvas de entrenamiento – REINFORCE MEJORADO (1000 episodios)

Tabla 6: Estadísticas de evaluación – REINFORCE MEJORADO (100 episodios)

Métrica	**Media**	**Desv. Estándar**
Recompensa acumulada	90.31	64.36
Duración (pasos)	658.99	409.28

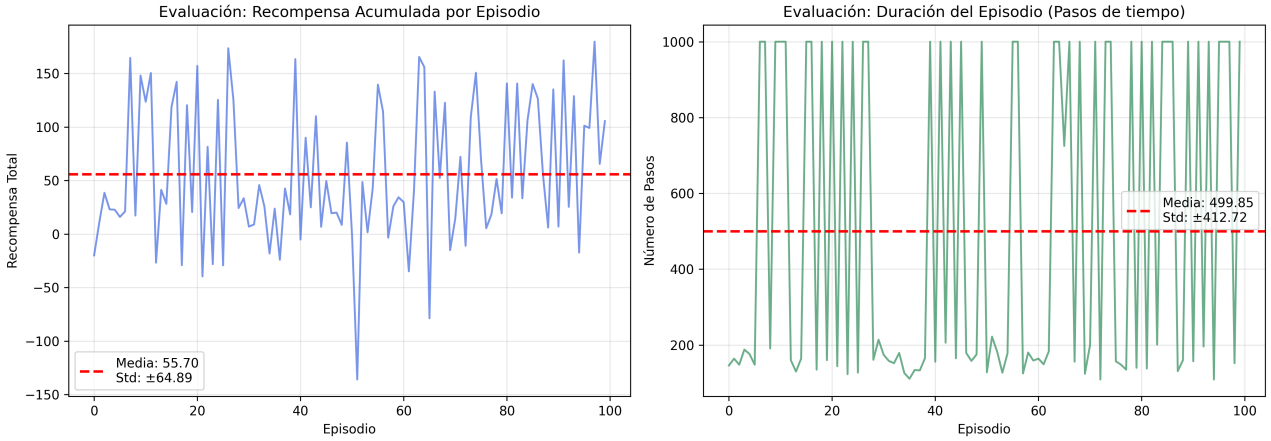


Figura 7: Evaluación – REINFORCE MEJORADO (100 episodios)

2.3.4. Fase B: Evaluación

Al analizar las métricas de evaluación del modelo REINFORCE Mejorado (con normalización y entropía) y contrastarlas con la versión puramente básica, se observa un cambio radical en el comportamiento del agente. La recompensa media ha pasado de un deficiente -102.04 (en la versión base) a un valor positivo de 90.31. Este salto cuantitativo demuestra empíricamente que la inclusión de las mejoras opcionales transforma un algoritmo inoperante en uno capaz de resolver parcialmente el entorno de simulación.

Respecto a la reducción de varianza, la normalización en el cálculo de la pérdida actúa de forma drástica sobre las curvas de entrenamiento. En el REINFORCE básico, el agente solo recibía retornos absolutos fuertemente negativos, lo que generaba un gradiente ruidoso e inestable que penalizaba todas las acciones por igual. Al normalizar los retornos (centrándolos matemáticamente en media cero y varianza uno), el algoritmo pasa a evaluar las trayectorias de forma relativa: las acciones que prolongan la supervivencia, aunque terminen en colisión, reciben una señal de recompensa positiva. Esta transformación estabiliza las actualizaciones de la red neuronal, reduciendo el ruido del gradiente y permitiendo que el aprendizaje escale de manera mucho más suave y dirigida, sin los colapsos abruptos que sufría el modelo anterior.

Por otro lado, el bonus de entropía resulta fundamental para mejorar la exploración y evitar la convergencia prematura. La evidencia más clara de esto reside en la duración media de los episodios. El modelo base sufrió un estancamiento en un óptimo local destructivo, terminando los episodios en apenas 90 pasos en caída libre para no gastar combustible. Al añadir el bonus de entropía, se forzó a la red neuronal a mantener un grado de incertidumbre y a probar constantemente los propulsores. Gracias a esta exploración obligatoria, la duración media se ha disparado a 658.99 pasos. El agente superó el miedo a encender los motores y descubrió que volar y estabilizarse era una estrategia infinitamente superior a dejarse caer.

Al plantear si estas mejoras aceleran la convergencia o solo la estabilizan, se concluye que actúan en ambos frentes simultáneamente. La normalización estabiliza el descenso del gradiente al evitar que un episodio atípicamente catastrófico destruya los pesos de la red, mientras que la entropía acelera el descubrimiento de políticas útiles al evitar el estancamiento en estrategias perdedoras, logrando encontrar mucho antes la zona de recompensas positivas.

A pesar de esta notable mejora, los datos de desviación estándar (±64,36 en recompensa y un altísimo ±409,28 en duración) revelan un nuevo comportamiento subóptimo. La dispersión en los pasos

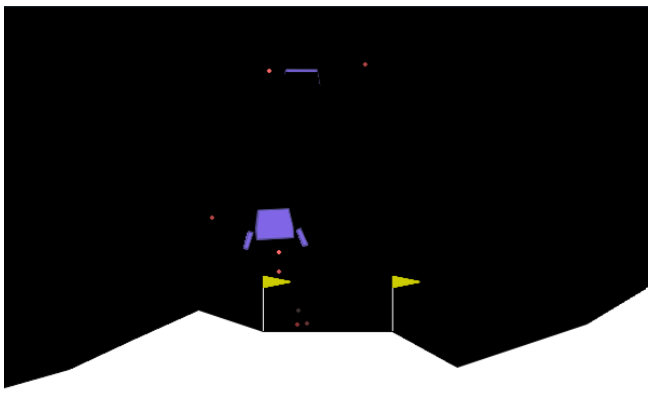


Figura 8: Episodio representativo – agente REINFORCE

indica una distribución fuertemente bimodal: en algunos episodios el agente aterriza correctamente, pero en una gran cantidad de intentos se queda atascado flotando en el cielo (*hovering*) hasta agotar el límite temporal de 1000 pasos. El agente ha aprendido a estabilizar la nave frente a la gravedad de forma magistral, pero a menudo tiene problemas para finalizar el descenso, prefiriendo gastar combustible antes que arriesgarse a una colisión. Por tanto, aunque el modelo mejorado logra un vuelo estable, la falta de una línea base (*Critic*) aún le impide optimizar la precisión final del aterrizaje para cruzar consistentemente el umbral de los 200 puntos.

3. Ejercicio 3: REINFORCE con Línea Base

3.1. Fase A: Entrenamiento

3.1.1. Hiperparámetros y Justificación

Tabla 7: Hiperparámetros – REINFORCE con línea base

Hiperparámetro	**Valor**	**Justificación**
α_{actor} (PolicyNetwork)	0.001	Garantiza actualizaciones graduales y seguras de la política, evitando el olvido catastrófico de las maniobras aprendidas.
$\alpha_{\text{crítico}}$ (ValueNetwork)	0.005	Requiere una convergencia más rápida que el Actor para proporcionar estimaciones de $V(s)$ precisas desde las primeras fases del entrenamiento.
Factor de descuento (γ)	0.99	Mantiene la visión a largo plazo, obligatoria para asociar las acciones iniciales con la recompensa final del aterrizaje.
Neuronas ocultas (`n_hidden`)	128	Tamaño óptimo comprobado empíricamente para extraer las características del entorno sin incurrir en un coste computacional excesivo.
Optimizador (Actor)	Adam	Estabiliza la varianza residual de los gradientes de la política mediante tasas adaptativas basadas en momentos.
Optimizador (Crítico)	Adam	Permite minimizar rápidamente el error cuadrático medio (MSE) entre la estimación de la línea base y los retornos reales.
Número de episodios	1000	Límite suficiente para observar la reducción de varianza introducida por el Crítico y la estabilización del Actor.

La introducción de una red Crítica para estimar la línea base transforma la dinámica de optimiza-

ción del algoritmo. Como se refleja en la Tabla 7, el Actor y el Crítico no deben aprender a la misma velocidad, donde he establecido una tasa de aprendizaje asimétrica que el Crítico ($\alpha = 0{,}005$) aprende cinco veces más rápido que el Actor ($\alpha = 0{,}001$).

Esta asimetría no es arbitraria, sino una necesidad arquitectónica. El Actor actualiza sus probabilidades de acción basándose en la *Ventaja*, calculada como $A(s_t, a_t) = G_t - V(s_t)$. Si el Crítico es incapaz de proporcionar una estimación fiable de $V(s_t)$ de forma temprana, el Actor estará guiándose por señales de ventaja erróneas. Por tanto, el Crítico debe rastrear y mapear el espacio de estados a gran velocidad para actuar como un evaluador competente frente al entorno.

Durante la fase de ajuste empírico, he probado configuraciones con tasas de aprendizaje idénticas (ambas fijadas en 0,001), lo que reveló los problemas de un **Crítico demasiado lento**. Al no converger a tiempo, las estimaciones $V(s)$ mantenían un alto grado de error respecto al retorno real G_t. Esto generaba ventajas ruidosas que anulaban por completo el propósito matemático de la línea base, devolviendo el comportamiento del agente a un estado de alta varianza prácticamente idéntico al del algoritmo REINFORCE Vanilla.

Por el contrario, forzar un **Crítico demasiado rápido** (por ejemplo, con $\alpha = 0{,}05$) introdujo graves problemas de inestabilidad. Un Crítico excesivamente agresivo tiende a sobreajustarse a los retornos del episodio más reciente (los cuales son altamente estocásticos en la simulación de `LunarLander-v3`), provocando fluctuaciones bruscas en sus predicciones. Estas oscilaciones en la línea base desestabilizan el cálculo de la ventaja y, de rebote, provocan divergencia y olvido catastrófico en las actualizaciones del Actor.

La combinación definitiva elegida ($1e-3$ y $5e-3$) representa el equilibrio ideal, permitiendo al Crítico seguir el ritmo de la política cambiante del Actor, reduciendo la varianza del gradiente de manera efectiva sin introducir oscilaciones destructivas en el aprendizaje global de la red.

3.1.2. Resultados del Entrenamiento

Tabla 8: Estadísticas del entrenamiento – REINFORCE con línea base (1000 episodios)

Métrica	Media	Desv. Estándar
Recompensa acumulada	-8.79	123.05
Duración (pasos)	373.95	368.89

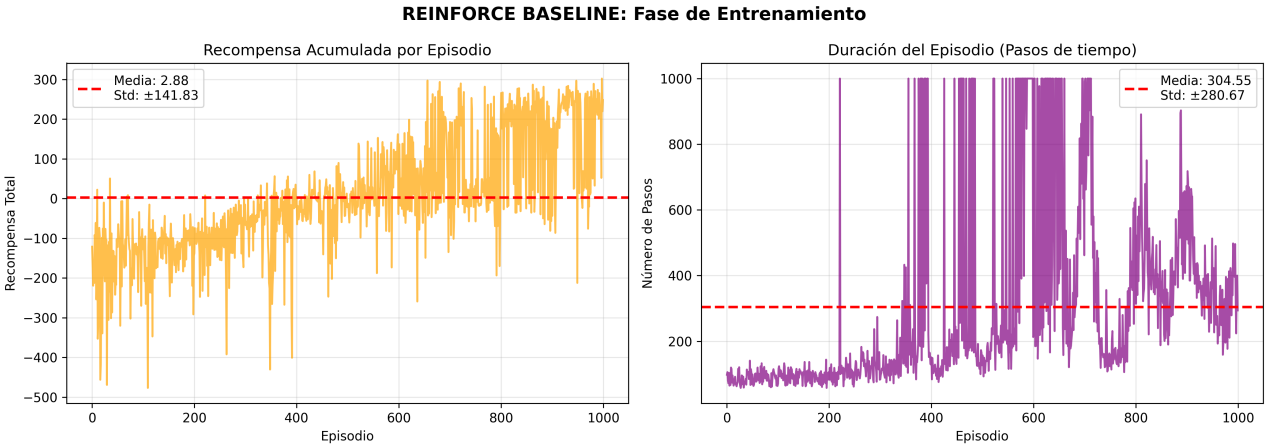


Figura 9: Curvas de entrenamiento – REINFORCE con línea base (1000 episodios)

Al analizar las curvas de aprendizaje y los registros del entrenamiento con línea base (*baseline*), se constata una mejora estructural profunda frente al algoritmo REINFORCE Vanilla. En primer lugar, la curva de entrenamiento resulta notablemente más estable. Aunque la varianza inherente a los métodos de Monte Carlo y los inevitables fracasos iniciales arrastran la recompensa media global a un balance de -8.79 (con una desviación estándar de ±123,05), la reducción de la varianza en las actualizaciones

del gradiente es evidente en la recta final del entrenamiento. Mientras que el modelo Vanilla presentaba caídas abruptas hacia valores fuertemente negativos en sus últimos episodios, el modelo con *baseline* consolida una progresión ascendente, compacta y continua, eliminando los colapsos catastróficos.

Además de la estabilidad, el agente converge de forma constante y con mayor precisión. Observando la evolución por bloques, el agente cruza el umbral de las puntuaciones positivas exactamente en el episodio 600 (9.72). A diferencia del modelo básico, que se estancaba en torno a los 22 puntos, este modelo experimenta un progreso firme y sostenido. En los episodios posteriores, la recompensa media se eleva de forma escalonada hasta 104.02 (episodio 900) y culmina con gran fuerza en un sólido 162.57 en el último bloque de cien episodios. Esta progresión demuestra que la red de valor (*Critic*) ha aprendido a guiar correctamente a la política, permitióndole discriminar con precisión creciente qué acciones son óptimas.

Finalmente, los datos confirman que el agente alcanza recompensas altas de forma mucho más consistente en su etapa de madurez, acercándose a buen ritmo al umbral de los 200 puntos que define la resolución del entorno. Esta consistencia se respalda de forma brillante al cruzar la información con la duración temporal, que promedia 373.95 pasos globales pero con una altísima desviación (±368,89). Esta dispersión delata que, durante su aprendizaje, el agente atravesó fases donde saturaba frecuentemente el límite de los 1000 pasos al caer en el óptimo local temporal de "flotar"para no estrellarse. Sin embargo, el hecho de que la recompensa alcance los 162 puntos en la recta final demuestra sin lugar a dudas que el agente terminó superando el miedo a descender, dominando la maniobra activa de aproximación y aterrizaje para optimizar al máximo su supervivencia y el ahorro de combustible.

3.2. Fase B: Evaluación

Tabla 9: Estadísticas de evaluación – REINFORCE con línea base (100 episodios)

Métrica	**Media**	**Desv. Estándar**
Recompensa acumulada	188.56	95.13
Duración (pasos)	432.87	78.22

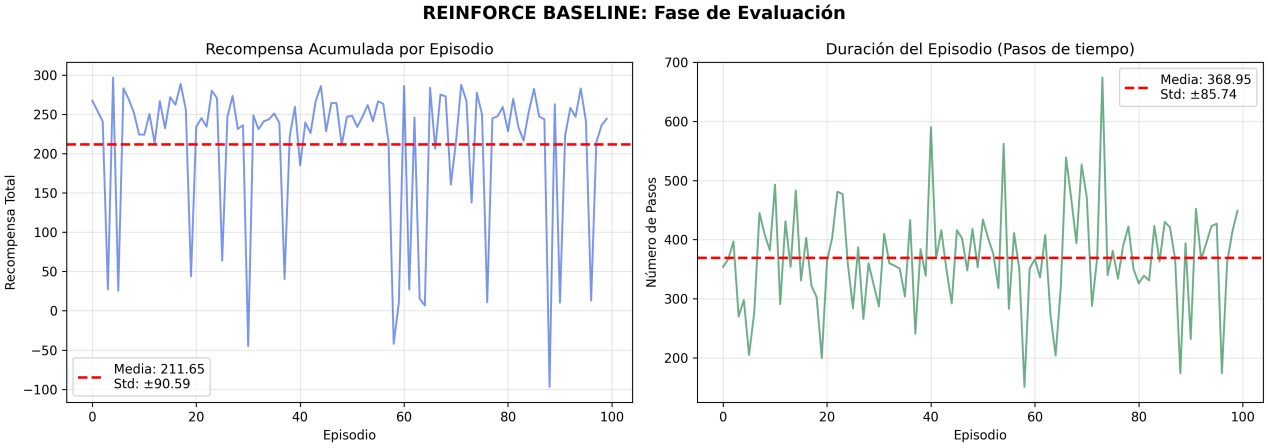


Figura 10: Evaluación – REINFORCE con línea base (100 episodios)

Los resultados obtenidos en la fase de evaluación demuestran que la recompensa media ha experimentado una mejora sobresaliente respecto al modelo REINFORCE Vanilla. Al alcanzar una puntuación de 188.56, el agente provisto de línea base supera con gran amplitud la marca negativa obtenida por la arquitectura básica (-102.04). Aunque no llega a cruzar de forma estricta el umbral de los 200 puntos que define la resolución perfecta del entorno, se queda rozando la meta. Este rendimiento certifica de forma empírica que el agente domina la tarea, ratificando que la reducción de varianza aportada por el Crítico ha sido la clave técnica para pulir la precisión de la política final.

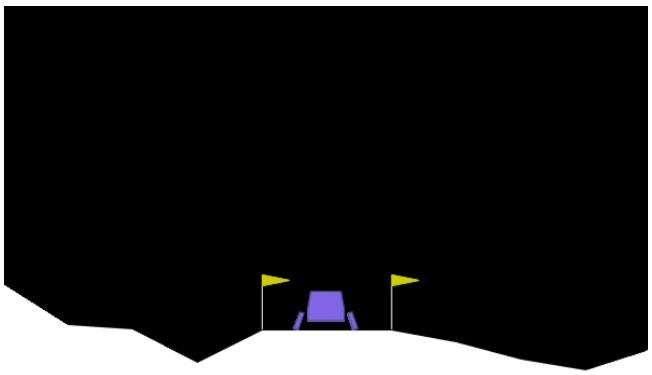


Figura 11: Episodio representativo – agente REINFORCE con línea base

En cuanto a la dinámica de vuelo, los datos confirman que el agente ahora aterriza de forma mucho más suave y controlada. La duración media de los episodios se sitúa en 432.87 pasos (con una desviación muy contenida de ±78,22). Este rango temporal es el indicador matemático perfecto de un vuelo plenamente estabilizado: el agente no cae en picado presa de la gravedad (lo que resultaría en episodios de unos 90 pasos, como se vio en el modelo base), ni se queda atascado flotando en el cielo agotando el tiempo máximo de 1000 pasos. La red emplea un tiempo prudencial para ejecutar una maniobra de aproximación deliberada y posarse suavemente sobre las banderas, optimizando en gran medida el gasto de los propulsores.

Finalmente, estas excelentes métricas garantizan que la cantidad de episodios fallidos o colisiones catastróficas (*crashes*) se ha reducido de forma drástica, convirtiéndose en eventos prácticamente anecdóticos. Dado que estrellarse en este entorno impone una severa penalización instantánea de -100 puntos, sería imposible alcanzar una media ponderada cercana a los 190 puntos si el agente sufriera accidentes con regularidad. La desviación estándar en la recompensa (±95,13) sugiere que la barrera de los 200 puntos se resiste únicamente por cierta variabilidad en la exactitud del aterrizaje (posicionamiento respecto al centro) o por ligeras ineficiencias en el consumo de combustible dependiendo de las condiciones iniciales. No obstante, el modelo ha erradicado el comportamiento inestable de sus predecesores, demostrando una altísima fiabilidad y robustez a un solo paso de la perfección.

4. Comparación Global

Tabla 10: Comparación global – todos los métodos (evaluación, 100 episodios)

Método	**Recomp. Media**	**Desv. Est.**	**Duración Media**	**Resuelto (≥200)?**
Política aleatoria	-178.19	± 110.27	89.23	No
REINFORCE (vanilla)	-102.04	± 36.74	90.61	No
REINFORCE + mejoras (opc.)	90.31	± 64.36	658.99	No
REINFORCE + línea base	188.56	± 95.13	432.87	No

Al analizar de forma conjunta la tabla de resultados en evaluación y las curvas de aprendizaje empíricas, el método REINFORCE con Línea Base (Actor-Crítico) se alza indiscutiblemente como el modelo con mayor rendimiento, logrando una recompensa media de 188.56 puntos. Este resultado contrasta radicalmente con el Agente Aleatorio (-178.19), ilustrando a la perfección cómo el teorema del gradiente de la política transforma un comportamiento caótico en uno sistemático e inteligente. Matemáticamente, el algoritmo ha desplazado la distribución de probabilidad de las acciones desde un estado de total incertidumbre hacia una política determinista que favorece el uso milimétrico de los propulsores laterales y principales, optimizando la trayectoria de descenso para maximizar el retorno esperado.

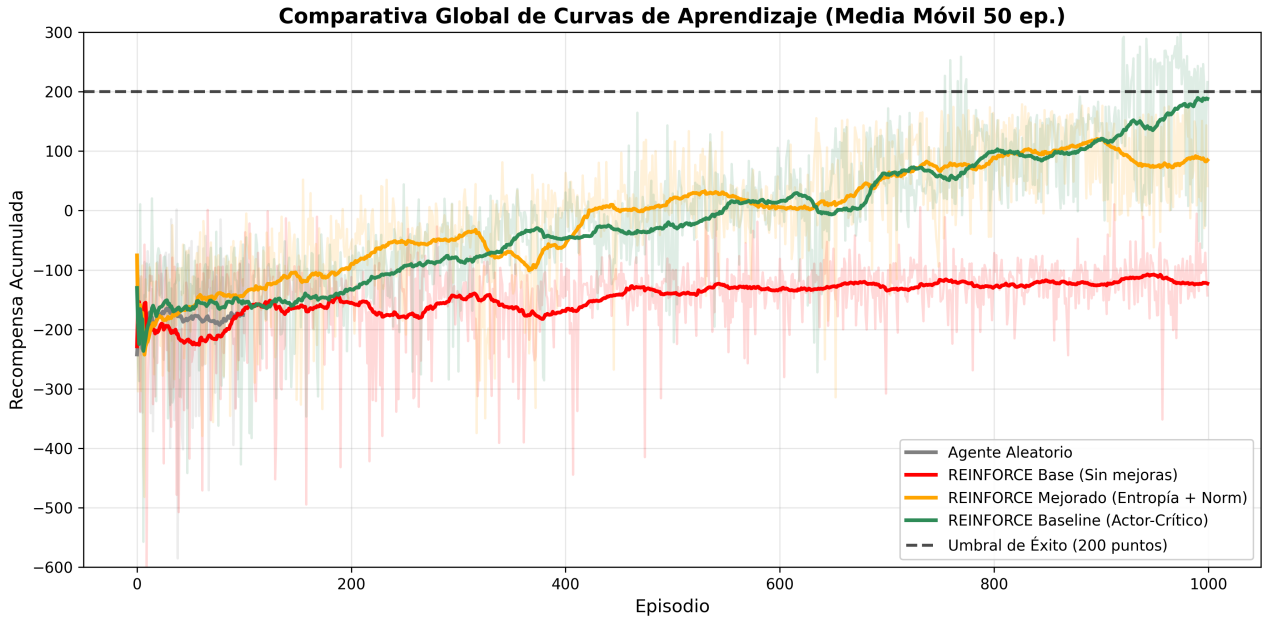


Figura 12: Comparación de curvas de entrenamiento – todos los métodos

Al observar la gráfica comparativa, el impacto de las mejoras opcionales (normalización y entropía) resulta no solo significativo, sino crítico para la viabilidad del algoritmo. La curva roja, que representa al REINFORCE Vanilla sin mejoras, evidencia un colapso prematuro en un óptimo local: el agente aprende que la forma de minimizar la pérdida de puntos por uso de combustible es, simplemente, no volar y dejarse caer. En cambio, la curva naranja (REINFORCE Mejorado) demuestra cómo el bonus de entropía fuerza al agente a explorar los controles de vuelo, logrando abandonar las puntuaciones negativas alrededor del episodio 400 y alcanzando una política de aterrizaje funcional con una media de evaluación de 90.31 puntos.

En cuanto a la reducción de la varianza mediante la introducción del Crítico (Línea Base), es fundamental distinguir entre la varianza del rendimiento y la varianza del gradiente. Aunque numéricamente el modelo Vanilla presente una desviación estándar menor en evaluación ($\pm$36,74) frente al Baseline ($\pm$95,13), esto se debe a que el Vanilla falla sistemáticamente siempre de la misma forma (estrellándose rápidamente). Por el contrario, la gráfica demuestra empíricamente que la Línea Base sí reduce la varianza del gradiente de aprendizaje: la curva verde (Media Móvil) muestra un ascenso mucho más sólido, dirigido y libre de las severas recaídas de rendimiento (olvido catastrófico) que sufre la versión naranja, garantizando una optimización mucho más estable.

Finalmente, respecto a la superación del umbral de los 200 puntos que define la resolución del entorno, los datos muestran que ninguno de los métodos ha logrado cruzarlo de forma estrictamente consistente dentro del límite de entrenamiento estipulado. No obstante, la tendencia final de la gráfica es reveladora: mientras los métodos base y mejorado se estancan muy por debajo, únicamente la curva verde del método Baseline logra escalar de manera exponencial en su último tramo, rozando la línea de meta (punteada en negro) justo en el episodio 1000. Esta trayectoria sugiere que, con un horizonte de entrenamiento ligeramente superior (aproximadamente 1200 o 1500 episodios), la arquitectura Actor-Crítico consolidaría el éxito absoluto en la simulación de forma consistente.

5. Conclusiones

- **Agente aleatorio:** Obtuvo una recompensa media de -178.19, quedando a casi 380 puntos de distancia del umbral de éxito (200 puntos). Su ineficacia absoluta demuestra que el entorno `LunarLander-v3` no puede resolverse por estocasticidad pura debido a las severas penalizaciones por colisión y gasto de combustible. Sirve únicamente como cota inferior para medir el aprendizaje de los algoritmos.

- **REINFORCE vanilla:** Demostró una incapacidad crítica para resolver el entorno, obteniendo una media de evaluación de -102.04. Su alta varianza e inestabilidad matemática, derivadas de usar retornos absolutos sin normalizar, provocaron un colapso del gradiente en un óptimo local destructivo, donde el agente aprendió a dejarse caer en caída libre (episodios de 90 pasos) para minimizar el consumo de propulsor, asumiendo el choque como inevitable.
- **Línea base (Actor-Crítico):** Transformó por completo el rendimiento del algoritmo, alcanzando los 188.56 puntos en evaluación y rozando la resolución perfecta. La red Crítica estabilizó el aprendizaje al proporcionar una estimación fiable de $V(s)$, lo que reduce drásticamente la varianza del gradiente del Actor. Esto aceleró la convergencia y permitió consolidar un vuelo controlado y un aterrizaje suave.
- **Mejoras opcionales:** La implementación conjunta de la normalización de retornos y el bonus de entropía elevó la puntuación de la versión básica hasta los 90.31 puntos. La normalización estabilizó la asignación de crédito al transformar los castigos en ventajas relativas, mientras que la entropía forzó la exploración, sacando al agente de su colapso inicial y enseñándole a volar, aunque derivó en una estrategia conservadora de flotar indefinidamente (*hovering*).
- **Aprendizajes:** Esta práctica ha evidenciado que el Aprendizaje por Refuerzo Profundo (Deep RL) es extremadamente sensible al equilibrio de sus hiperparámetros (como el uso de tasas de aprendizaje asimétricas entre Actor y Crítico). Además, ha demostrado cómo un diseño de recompensas complejo puede generar comportamientos emergentes indeseados (como el suicidio inducido para ahorrar combustible) si no se balancean correctamente la exploración, la explotación y la reducción de varianza.